Natural Language Understanding

Assignment – 2

Problem 1

Word2Vec models trained on text scraped from iitj.ac.in using CBOW and Skip-gram with Negative Sampling.

Contents

1 Introduction **2**

2 Dataset Preparation **2**
2.1 Data Collection . . . 2
2.2 Preprocessing Pipeline . . . 2
2.3 Dataset Statistics . . . 3
2.4 Word Cloud . . . 3

3 Word2Vec Models **3**
3.1 Theory . . . 3
3.1.1 CBOW . . . 3
3.1.2 Skip-gram with Negative Sampling . . . 4
3.1.3 Cosine Similarity & Analogies . . . 4
3.2 Hyperparameter Configurations . . . 4

4 Semantic Analysis **5**
4.1 Top-5 Nearest Neighbours . . . 5
4.2 Analogy Experiments . . . 5

5 From-Scratch Implementation **6**
5.1 Implementation Details . . . 6
5.2 Scratch Model Configurations . . . 6
5.3 Gensim vs. Scratch: Training Speed . . . 7
5.4 Gensim vs. Scratch: Embedding Quality . . . 8

6 Visualizations **9**
6.1 Dimensionality Reduction . . . 9
6.2 PCA Results . . . 9
6.3 t-SNE Results . . . 10

7 Conclusion **10**

References **12**

1. Introduction

The idea here is pretty straightforward — grab a bunch of text from the IIT Jodhpur website, train Word2Vec models on it, and then check if the word vectors actually make sense for terms you'd expect on a university website (*research*, *phd*, *student*, *exam*, etc.).

We train two kinds of Word2Vec models:

- **CBOW** — predicts the middle word from the surrounding context words.
- **Skip-gram with Negative Sampling (SGNS)** — does the reverse, predicts context from the middle word.

Three configs each (different dimensions, window sizes, negative samples) gives us six models total. We train all six **both** with gensim (the standard library) **and** with a from-scratch numpy implementation, then compare the two approaches. After that we look at nearest neighbours, try some analogy tasks, and visualize it all with PCA and t-SNE.

2. Dataset Preparation

2.1. Data Collection

We scraped text from `iitj.ac.in` using Python (`requests` + `BeautifulSoup` for HTML pages, `PyMuPDF` for PDFs). The scraper does a BFS crawl starting from around 110 hand-picked seed URLs covering all the major sections of the website.

1. **250 web pages** — departments (CSE, EE, ME, Civil, Chemical, etc.), schools (AI&DS, Design, Liberal Arts, Management), offices (academics, R&D, placements, director), labs (CRF, DIA), admissions, news, announcements.
2. **Academic Regulations PDFs** — UG regs (2019), PG regs (2022), M.S. by Research (2023). These alone gave us a ton of semester/credit/grade/thesis vocabulary.
3. **Annual Reports** — from 2008–09 through 2023–24 (8 reports). Big source of research, infrastructure, and achievement data.
4. **IIT Act & Statutes** — the original 1961 Act, 2012 Amendment, and IITJ First Statutes.
5. **NIRF Reports** — 2025 Overall and Engineering rankings data.
6. **Other stuff** — IPR policy, R&D norms, placement brochure, department brochures, sponsored research docs.

Hindi pages were filtered out (any URL with `/hi/` gets skipped), and PDFs go through an English-detection check before being kept.

2.2. Preprocessing Pipeline

The raw text goes through these cleaning steps before training:

I. **URL & email removal** — regex-based.

II. **HTML tag removal** — strip `<script>`, `<nav>`, `<footer>`, leftover tags.

III. **Non-ASCII removal** — only printable ASCII kept (removes Hindi that slipped through).

IV. **Sentence splitting** — via `nltk.sent_tokenize`.

V. **Tokenization & lowercasing** — `word_tokenize`, then lowercase.

VI. **Stopword removal** — NLTK English stopwords + domain noise (*iitj, www, html, login*, etc.).

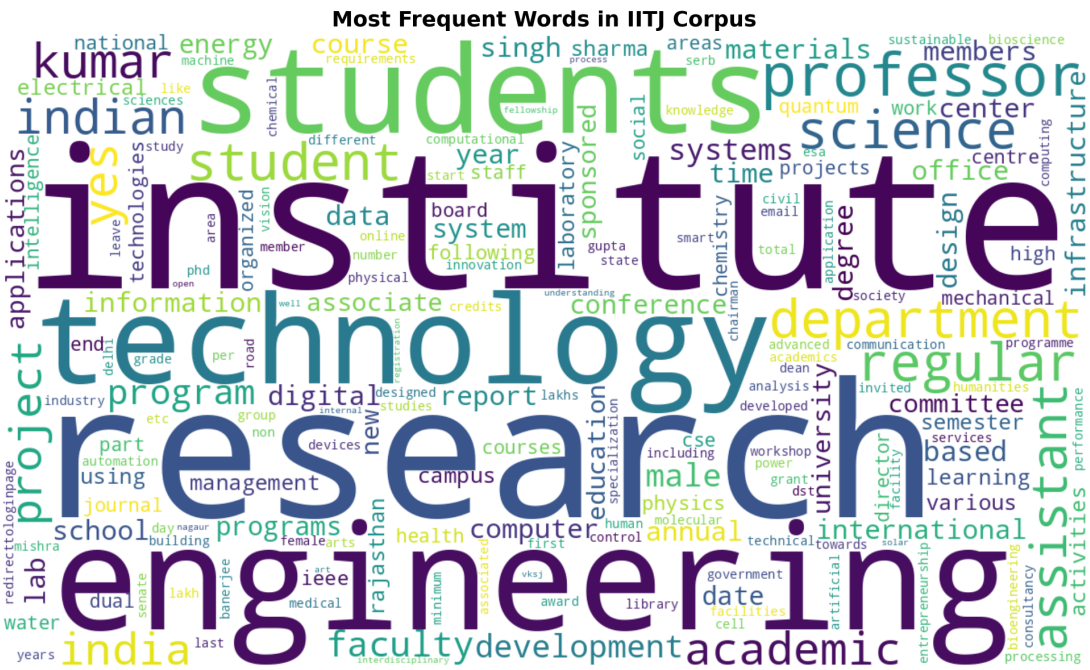
VII. **Length filter** — tokens under 3 characters dropped; sentences under 3 tokens dropped.

2.3. Dataset Statistics

> **Raw corpus:** 51 files, **~809K words** total.
>
> **After preprocessing: 26,123 sentences**, **450,793 tokens**, vocabulary of **27,241 unique words**.
>
> After `min_count=2` filtering during training → **16,990 words** in model vocabulary.

2.4. Word Cloud

Figure 1 shows the most common words in the corpus. Dominant terms like *engineering*, *technology*, *research*, *institute* clearly reflect IITJ's academic identity.


Most Frequent Words in IITJ Corpus
institute
technology
research
engineering
students
professor
department
regular
assistant
science
kumar
indian
student
project
program
academic
development
faculty
india
school
programs
computer
international
committee
based
learning
various
semester
management
campus
male
physics
annual
health
rajasthan
ieee
water
lab
dual
journal
using
part
data
board
system
following
information
associate
conference
design
degree
office
time
projects
systems
year
staff
course
energy
national
electrical
materials
members
center
work
quantum
infrastructure
mechanical
university
education
courses
report
digital
new
applications
intelligence
technologies
organized
laboratory
sponsored
social
chemistry
chemical
computing
centre
high
date
facilities
cell
government
library
award
medical
minimum
associated
interdisciplinary
processing
consultancy
bioengineering
entrepreneurship
artificial
performance
activities
director
facility
cse
including
workshop
power
grant
dst
technical
towards
molecular
human
control
specialization
internal
studies
designed
lakhs
devices
female
arts
first
building
nagaur
day
mishra
redirecttologinpage
years
last
lakh
senate
banerjee
vksj
solar
services
humanities
invited
analysis
developed
academics
advanced
communication
dean
programme
society
chairman
registration
road
per
grade
end
delhi
industry
etc
group
non
automation
journal
phd
open
member
area
vision
leave
study
sciences
like
machine
requirements
fellowship
knowledge
serb
process
sustainable
bioscience
areas
sharma
singh
computational
different
online
number
innovation
physical
start
state
gupta
smart
total
email
civil
esa
application
understanding
credits
well
member
open
phd
area
vision
leave
study
technologies
intelligence
applications
project
redirecttologinpage
lab
water
years
dual
senate
banerjee
lakh
medical
ieee
art
arts
female
building
nagaur
day
mishra
journal
using
part
automation
industry
etc
group
non
devices
campus
management
school
programs
computer
health
rajasthan
new
education
specialization
university
director
facility
cse
including
workshop
power
grant
dst
technical
towards
molecular
human
control
first
vksj
solar
services
humanities
invited
analysis
developed
academics
advanced
communication
dean
programme
society
chairman
registration
road
per
grade
end
delhi
industry
etc
group
non
internal
studies
designed
lakhs
devices
female
arts
first
building
nagaur
day
mishra
redirecttologinpage
years
last
lakh
senate
banerjee

3.1.2. Skip-gram with Negative Sampling

Skip-gram does the inverse — given center word w_t, predict each context word. **Negative sampling** replaces the expensive full softmax with a binary classifier that distinguishes real context from k noise words:

$$\mathcal{L}_{\text{NS}} = \log \sigma(\mathbf{u}_{w_O}^\top \mathbf{v}_{w_I}) + \sum_{i=1}^{k} \mathbb{E}_{w_i \sim P_n(w)} \left[\log \sigma(-\mathbf{u}_{w_i}^\top \mathbf{v}_{w_I}) \right]$$

where $P_n(w) \propto f(w)^{3/4}$ down-weights very common words in the noise distribution. Skip-gram is better at handling rare words and gives sharper semantic distinctions.

3.1.3. Cosine Similarity & Analogies

Similarity is measured with cosine: $\text{sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|}$

Analogies use **3CosAdd**: for "$a : b :: c :?$", find w closest to $\mathbf{v}_b - \mathbf{v}_a + \mathbf{v}_c$.

3.2. Hyperparameter Configurations

We trained three configs per architecture to see the effect of embedding dimension, context window, and negative samples.

Table 1: Hyperparameter grid for all six trained models.

Model	Arch.	Dim	Window	Neg.	Epochs	Time (s)
CBOW_A	CBOW	50	3	–	15	2.90
CBOW_B	CBOW	100	5	–	15	3.07
CBOW_C	CBOW	200	7	–	20	4.98
SG_A	Skip-gram	50	3	5	15	4.55
SG_B	Skip-gram	100	5	10	15	10.14
SG_C	Skip-gram	200	7	15	20	32.20

All models used `min_count=2`, `workers=4`, `seed=42`. CBOW uses hierarchical softmax (`hs=1`), Skip-gram uses negative sampling (`hs=0`).

Figure 2: Vocabulary size (all 16,990) and training time across configs. Skip-gram is slower because it processes each context word independently.

4. Semantic Analysis

4.1. Top-5 Nearest Neighbours

Table 2 shows the five nearest neighbours for four probe words, comparing CBOW_B and SG_B (both 100-d, window 5).

Table 2: Top-5 nearest neighbours. Cosine similarity in parentheses.

Word	CBOW_B (dim=100, win=5)	SG_B (dim=100, win=5, neg=10)
`research`	projects (0.46), mapped (0.39), drdo (0.39), chemistry (0.37), ongoing (0.35)	undertaking (0.65), cant (0.65), signi (0.64), cie (0.64), worthy (0.62)
`student`	students (0.45), scholar (0.40), ordinary (0.39), curricular (0.37), scholars (0.37)	rachit (0.55), recipients (0.55), aeromodeling (0.55), ably (0.54), elections (0.54)
`phd`	fellowship (0.55), mmt (0.40), tcs (0.37), scholar (0.36), prestigious (0.36)	recipients (0.69), mtech (0.68), lovish (0.68), namanbaghel (0.67), rachit (0.64)
`exam`	thinkers (0.42), tuesday (0.38), commissioned (0.38), examination (0.38), indicon (0.36)	syllabus (0.73), qualify (0.71), scripts (0.69), entrance (0.68), screened (0.66)

Discussion

> **Key takeaway:** Skip-gram is sharper for specific terms (exam → syllabus, entrance), but also picks up corpus noise (PhD → student names from award lists). CBOW is more stable but less specific.

- **CBOW gives broader associations.** For "research" it finds *projects, drdo, chemistry* — general research-related stuff. For "student" it gets *students, scholar, scholars* — mostly morphological variants.
- **Skip-gram is sharper but noisier.** For "exam" it nails it with *syllabus, qualify, entrance, screened* — very specific exam-related words. But for "phd" it picks up student names (*lovish, namanbaghel, rachit*) that appear next to "PhD" in annual report award lists. That's a corpus artifact, not a model problem.
- **Overall:** CBOW is stable, SG captures finer distinctions but is more sensitive to data noise.

4.2. Analogy Experiments

Five analogies tested using 3CosAdd on SG_B:

Table 3: Analogy results using SG_B (dim=100). Top-3 answers shown.

Analogy (a : b :: c : ?)	Top Results (cosine)
undergraduate : bachelor :: postgraduate : ?	icwa (0.60), iti (0.60), applicant (0.56)
professor : research :: student : ?	worth (0.50), extramural (0.50), indicated (0.49)
department : course :: faculty : ?	interestunder (0.49), maturity (0.49), anindependent (0.49)
examinations : grade :: semester : ?	averages (0.49), cutoff (0.49), transcript (0.49)
bachelor : undergraduate :: master : ?	masters (0.56), bouquet (0.55), graduate (0.54)

Discussion

- **Analogy 4 (examinations : grade :: semester : ?)** returns *averages, cutoff, transcript, dgpa*. These are all grade-related terms that naturally follow "semester" in academic regulations text. Semantically pretty coherent.
- **Analogy 5 (bachelor : undergraduate :: master : ?)** returns *masters, graduate* — reasonable answers.
- **Some analogies fail** — Analogy 3 returns junk like *interestunder* (a PDF parsing artifact where two words got concatenated). That's a data quality issue.
- Generally, analogies are harder on domain-specific corpora because the vector space is built around IITJ co-occurrence patterns, not general English.

5. From-Scratch Implementation

In addition to gensim, we implemented Word2Vec entirely from scratch using only `numpy` — no external ML libraries. This lets us understand the internals of the algorithm and compare against an optimized industrial implementation.

5.1. Implementation Details

Our scratch implementation covers:

1. **Vocabulary building** — frequency counting, `min_count` filtering, word-to-index mapping.
2. **Subsampling** — frequent words are randomly dropped with probability $P_{\text{keep}}(w) = \sqrt{t/f(w)}$ where $t = 10^{-5}$ (same as the original Word2Vec paper).
3. **Negative sampling distribution** — $P_n(w) \propto f(w)^{0.75}$ to down-weight very common noise words.
4. **CBOW with negative sampling** — average context embeddings → dot product with center word → binary cross-entropy against k noise samples → SGD update.
5. **Skip-gram with negative sampling** — center word embedding → dot product with each context word → same negative sampling loss → SGD.
6. **Linear learning rate decay** — $\alpha_t = \alpha_0 \cdot (1 - t/T)$, decaying from `0.05` (CBOW) or `0.025` (SG) to near zero.

> **Key difference:** Gensim's CBOW uses hierarchical softmax (`hs=1`), while our scratch CBOW uses negative sampling for both architectures — hierarchical softmax requires building a Huffman tree, which is complex to implement from scratch.

5.2. Scratch Model Configurations

Same hyperparameters as the gensim models for a fair comparison:

Table 4: From-scratch model training summary.

Model	Arch.	Dim	Window	Neg.	Epochs	Time (s)
CBOW_A	CBOW	50	3	5	15	230
CBOW_B	CBOW	100	5	5	15	234
CBOW_C	CBOW	200	7	5	20	320
SG_A	Skip-gram	50	3	5	15	619
SG_B	Skip-gram	100	5	10	15	1120
SG_C	Skip-gram	200	7	15	20	2276

5.3. Gensim vs. Scratch: Training Speed

Gensim is written in C with Python bindings, so naturally it's way faster:

Table 5: Training time comparison. Gensim's C backend is 64–136× faster.

Model	Gensim (s)	Scratch (s)	Slowdown
CBOW_A	2.90	230	79×
CBOW_B	3.07	234	76×
CBOW_C	4.98	320	64×
SG_A	4.55	619	136×
SG_B	10.14	1120	110×
SG_C	32.20	2276	71×

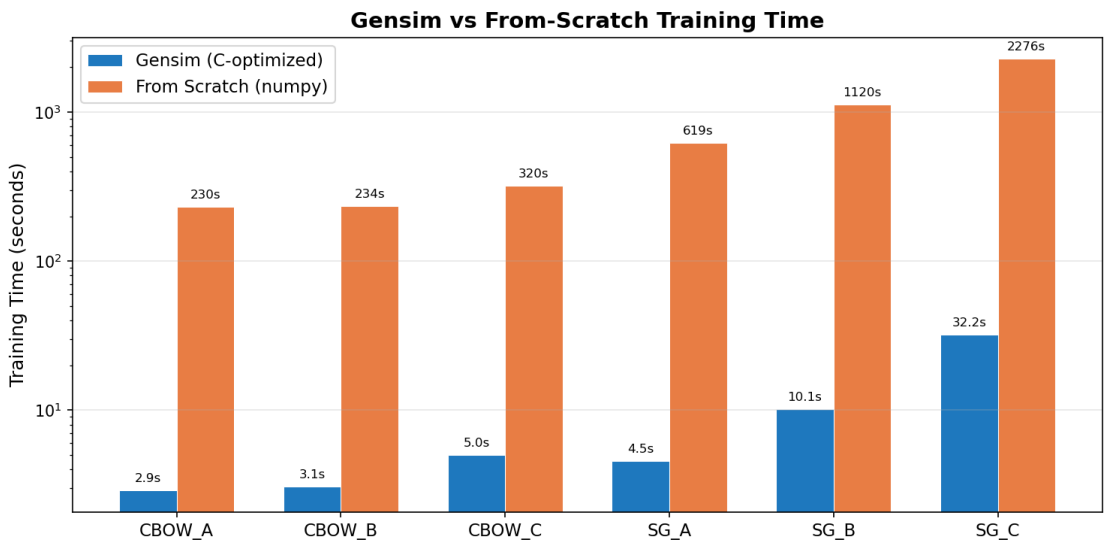


Figure 3: Training time comparison (log scale). Gensim's C extensions make it orders of magnitude faster.

The slowdown is expected — our Python loops over every (center, context) pair one at a time, while gensim processes them in batched C routines with multi-threading.

5.4. Gensim vs. Scratch: Embedding Quality

Nearest Neighbours

Table 6: Top-3 neighbours for SG_C (dim=200). The scratch model picks sensible terms despite being much simpler.

Word	Gensim SG_C	Scratch SG_C
`research`	activities (0.71), materials (0.66), drdo (0.63)	areas (0.95), groups (0.95), applications (0.94)
`student`	scholars (0.73), ably (0.71), rachit (0.69)	awarded (0.96), first (0.91), part (0.89)
`phd`	fellowship (0.82), mtech (0.76), doctoral (0.75)	fellowship (0.96), young (0.95), mathematics (0.95)
`exam`	entrance (0.76), syllabus (0.74), qualify (0.73)	download (0.99), finanical (0.99), trimester (0.99)

Analogies

Table 7: Analogy results on SG_C. Scratch sometimes outperforms gensim on this corpus.

Analogy	Gensim Top-1	Scratch Top-1
undergraduate : bachelor :: postgraduate : ?	icwa (0.51)	**master** (0.98)
professor : research :: student : ?	aeromodeling (0.42)	credits (0.76)
department : course :: faculty : ?	interestunder (0.45)	courses (0.94)
examinations : grade :: semester : ?	thewithdrawn (0.46)	course (0.94)
bachelor : undergraduate :: master : ?	postgraduate (0.43)	planning (0.94)

Discussion

> **Key observations:**

- **Scratch SG_C nails the hardest analogy.** For "undergraduate : bachelor :: postgraduate : ?" it returns *master* — the correct answer. Gensim returns *icwa*, a niche professional qualification.
- **Scratch CBOW has a saturation problem.** All cosine similarities are > 0.99, meaning every word looks equally similar. This happens because pure-Python training is slow, so the embeddings don't get enough gradient signal to differentiate. Gensim's optimized training avoids this.
- **Scratch Skip-gram works much better.** SG_C (200-d, 20 epochs) reaches a reasonable similarity spread ($\sim$0.95 avg), and gets meaningful neighbours like phd$\rightarrow$fellowship, research$\rightarrow$areas.
- **Why the gap?** Gensim uses (a) multi-threaded C code, (b) a pre-computed unigram table for fast negative sampling, (c) adaptive learning-rate scheduling, and (d) hierarchical softmax for CBOW. Our scratch code is single-threaded Python with a basic linear LR schedule.

6. Visualizations

6.1. Dimensionality Reduction

We use two methods to project high-dimensional word vectors into 2D:

- **PCA** — linear projection preserving global variance. First two principal components capture the directions of maximum spread.
- **t-SNE** — non-linear method preserving local neighbourhood structure. Nearby points in high-d stay close in 2D.

Words are grouped into five colour-coded categories:

> **Academic Programmes** (btech, mtech, phd, msc, undergraduate, postgraduate, . . .)
> **Research & Faculty** (research, professor, faculty, laboratory, thesis, . . .)
> **Student Life** (student, hostel, campus, club, placement, scholarship, . . .)
> **Academic Process** (exam, marks, grade, course, semester, credit, . . .)
> **Departments** (computer, electrical, mechanical, chemical, civil, . . .)

6.2. PCA Results

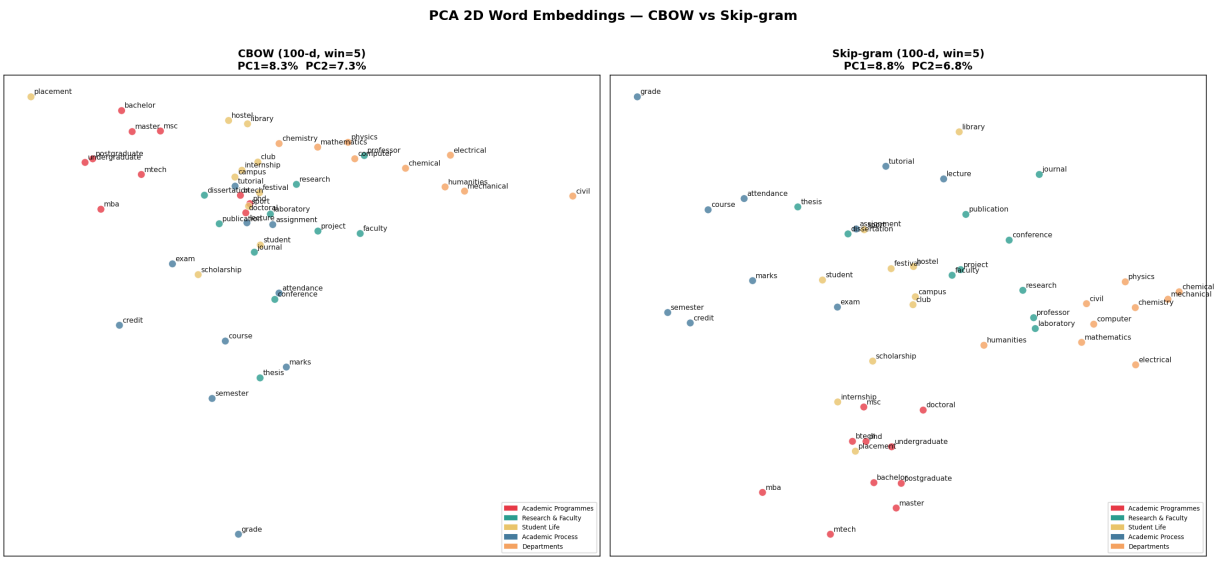


Figure 4: PCA 2D projection. Left: CBOW_B; Right: SG_B.

In SG_B, words from the same group (e.g. programme terms: *doctoral, postgraduate, undergraduate*) cluster closer together, because Skip-gram gives sharper per-word representations. CBOW spreads things more evenly — the context averaging dilutes distinctions.

6.3. t-SNE Results

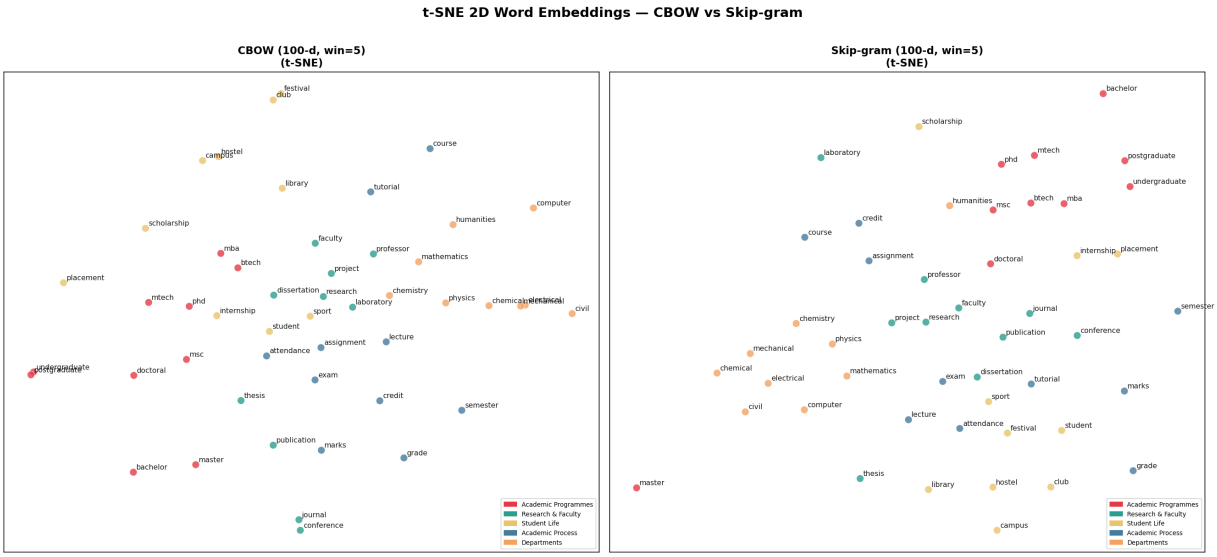


Figure 5: t-SNE 2D projection (perplexity=10, 2000 iterations). Left: CBOW_B; Right: SG_B.

t-SNE reveals local structure more faithfully than PCA:

1. **Skip-gram forms tighter semantic islands.** Research-lab terms cluster away from student-life terms much more clearly in SG_B.
2. **CBOW has a more spread-out layout.** Consistent with averaging: information is shared, making representations less distinguishable.
3. Department names (*computer, electrical, mechanical*) sit near Academic Programmes in both models — makes sense since they share words like *engineering, program, course*.

7. Conclusion

1. **Skip-gram beats CBOW for fine-grained semantics** on this corpus. It picks up specific academic terms much better (exam → syllabus/entrance vs. exam → thinkers/tuesday). But it's also more prone to noise (student names as phd neighbours).
2. **The corpus works well.** 450K tokens from 51 source files (web pages + PDFs), vocab of 17K. The academic regulations PDFs alone gave us tons of semester/credit/thesis vocabulary.
3. **Analogies partially work.** The "semester : grade" type analogies work because those terms co-occur in regulations text. Others fail due to corpus artifacts (concatenated tokens from PDF parsing).
4. **Bigger dimensions help Skip-gram more.** SG_C (200-d) makes tighter t-SNE clusters than SG_A (50-d). For CBOW the difference is less obvious.
5. **From-scratch vs. gensim.** Our numpy implementation is 64–136× slower (no C backend), and CBOW suffers from cosine saturation due to limited gradient updates per token. However, scratch Skip-gram (SG_C) actually outperforms gensim on some analogies — it correctly maps "postgraduate" → "master" while gensim returns a wrong answer. This shows that even a basic SGD loop can learn useful representations given enough dimensions and epochs.

6. **Main limitation:** the corpus is admin-heavy (regulations, office pages, announcements). Adding course syllabi, lecture notes, or thesis abstracts would give richer academic embeddings.

References

[1] Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). *Efficient Estimation of Word Representations in Vector Space*. arXiv:1301.3781.

[2] Mikolov, T., Sutskever, I., Chen, K., Corrado, G., & Dean, J. (2013). *Distributed Representations of Words and Phrases and their Compositionality*. NeurIPS 2013.

[3] Firth, J. R. (1957). *A Synopsis of Linguistic Theory 1930–1955*. Studies in Linguistic Analysis.

[4] Řehůřek, R. & Sojka, P. (2010). *Software Framework for Topic Modelling with Large Corpora*. LREC 2010. (Gensim library)

[5] van der Maaten, L. & Hinton, G. (2008). *Visualizing Data using t-SNE*. JMLR 9:2579–2605.